Date: . . . . No: . . . .

Problem 1: 2,0
Problem 2: 2,5
Problem 3: ~~1,25~~ 1,5
Problem 4: 2,5 (-1,65)
~~7,75~~
8,35

Problem 1: 2,1
Problem 2 2,5
Problem 3 1,75
Problem 4 2,5
8,85

MSSV: N25PECE070

Họ và tên: Nguyễn Quốc Toàn

Problem 1: Singly linked list – In Place Reversal & Pointer tracing

Given a singly linked list: head → [10] → [25] → [20] → [15] → NULL.

- Task 1: Write a function / algorithm in pseudocode to reverse the singly linked list, including three pointers to complete your function.
- Task 2: ~~trace the execution steps (via text description or pointer diagrams) when inserting newValue = 5 into the given list~~
- Task 2: Draw a diagram or step by step pointer trace table showing the position and values of the three pointer after each iteration until the list is fully reversed

+ Solution are answers:

1,1 Task 1:

Code:

```
function Reverse( head):
    prev ← NULL.
    curr ← head.
    next ← NULL
    while ( curr ≠ NULL ):
        next ← curr.next.
        curr.next ← prev.
            prev ← curr
            curr ← next.
    return prev.
```

~~0,75~~ 1,00 Task 2:

Thành Đạt

Step	prev	curr	next
init	NULL	10	-
1	10	25	25
2	25	40	40
3	40	15	15
4	15	NULL	NULL

Linked Structure

?

-0,5

- Loop Stop because curr = NULL. New head = prev = 15.

Result: head → 15 → 40 → 25 → 10 → NULL.

~~Problem 2: circular linked list - insert at head.~~

~~Given a Strong singly circular linked list: head → [10] → [20] → [30] → (points back to head)~~

~~• Task 1: write a pseudocode for the function InsertHead(head, NewValue) to insert a new node containing new value immediately before the first occurrence of target.~~

~~• Task 2: list the exact order of pointer assignment required and briefly justify why this specific sequence avoids memory leaks or broken links~~

~~• Task 3: Explicitly handle the following three edge cases in code/pseudocode.~~

~~1. target is located at the head node.~~

0.5 Problem 2: circular linked list - insert at head.

• Given a Singly circular linked list: head → [10] → [20] → [30] → (point back to head)

• Task 1: write a pseudocode for the function insertHead(head, NewValue) to insert a new node containing new value immediately before the first occurrence of target

• task 2: trace the execution step (via text description or pointer diagram) when inserting new value 5 into the given list.

~~Answer~~!

Answer!

1,25 task 1:

```
function Inserthead (Head, newvalue):
    node ← new Node (newvalue).
    If (Head = NULL):
        node.next ← node.
        return node.
    tail ← head.
    while (tail.next ≠ head):
        tail ← tail.next.
    node.next ← head.
    tail.next ← node.
    return node.
```

1,25 task 2!

Step	Action	Result:
1	create node(5)	node = 5
2	tail ~~n~~ = head	tail = 10
3	tail.next = 20 ≠ head.	tail = 20
4	tail.next = 30 ≠ head.	tail = 30
5	tail.next = 10 = head.	~~tail~~ Stop.
6	~~node~~ node.next = head	5→10
7	tail.next = node.	30→5
8	head = node.	head = 5

1,5
~~1,25~~

Problem 3: Doubly linked list – insert before target.

Given a doubly linked list containing integers.

· task 1: write pseudocode for insertBefore (head, target, newvalue)

Thành Đạt

to insert a new node containing newValue immediately before the first occurrence of target.

- task1: list the exact order of pointer assignments required and brief briefly. Explain why this specific sequence avoid memory leak or broken links.
- task2: Explicitly handle the following two edge cases, visible in code/pseudocode.
  1. target is located at the head node (head)
  2. target doesn't exist in the list

Answer:

```
function InsertBefore(head, target, newValue):
        curr ← head
    while curr ≠ NULL and curr.data ≠ target:
        curr ← curr.next
    If curr = NULL:
        return head
    node ← new Node(newValue)
    ~~If curr.prev prev ← NULL~~
                node.
    If curr = head:
        node.prev ← NULL
        node.next ← head
        head.prev ← node
        return node
    node.prev ← curr.prev
    node.next ← curr
    curr.prev.next ← node
    curr.prev ← node
    return head
```

MSSV: N25DECE070

Date: . . . . No: . . . .

Họ và tên: Nguyễn Quốc Toàn

task2: general case: (1) → (2) → (3) → (4)

Why this order is safe:

0,00

?

- (1) and (2) only set the new node's pointers, so the existing list is not changed yet.
- (3) uses curr.prev to reach the old ~~prevous~~ previous node, so it must run before (4)
- If (4) ran first, curr.prev would already be node. Then (3) would make node.next = node and the old previous node would never link to the new node. The list would be broken and the old link lost

0,25 task1:

1,75

```
function InsertBefore(Head, target, newvalue):
    curr ← head
    while (curr ≠ NULL ~~and~~ and curr.data ≠ target)
        curr ← curr.next
    node ← new Node(newvalue)
        node.prev ← curr.prev
        node.next ← curr
        curr.prev.next ← node
        curr.prev ← node
    return ~~return~~ head
```

task3:

0,75

```
function insertBefore(head, target, newvalue):
    curr ← head
    while (curr ≠ NULL and curr.data ≠ target:)
        curr ← curr.next
    if (curr = NULL)
        return head
```

Thành Đạt

Scanned with CamScanner

```
node ← newnode. node (newValue).
  If curr = head:
      node.prev ← NULL.
      node.next ← head
      node.prev ← node.   1/2
      return node.
  node.prev ← curr.prev.
  node.next ← curr
  ~~node~~. curr.prev.next ← node.
  ~~cu~~ curr.prev ← node
  return head.
```

2.5 Problem 4. Sorted linked list - In order. Insertion.

Given a singly linked list sorted in ascending order head → [1] → [5] → [8] → NULL.

task 1: write the function ~~sort~~ SortedInsert (head, val) to insert val into the list such that the ascending order is preserved.

task 2: constructs a step by step trace table detailing pointers iterations and condition evaluations when inserting val=6 into the list

Answer:

1.5 task 1:

```
function SortedInsert (head, val):
    node ← ~~the~~ new Node(val).
    If (head = NULL or val < head.data:)
        node.next ← head
        return. node.
```

```
curr ← head.
while ~~curr head~~ (curr.next ≠ NULL and curr.next.data < val):
        curr ← curr.next.
    node.next ← curr.next.
    curr.next ← node.
    return head
```

1.25

Task 2:

Step	Curr	Curr.next	Condition	Result:
Init	2	5	head = NULL or 6<2?	False → goto loop
1	2	5	5 ≠ NULL and 5<6	True → curr=5
2	5	8	8 ≠ NULL and 8<6	False → Stop
3	5	8	node.next=8	6 → 8
4	5	6	Curr.next=node.	5 → 6

Result: head → 2 → 5 → 6 → 8 → NULL